Reproduce-then-Extend — Snake Habitat Selection (Ecology) (Day 2)

Intro to Machine Learning · Bruce A. Desmarais

July 03, 2026

Published study. Multiscale spatial model of snake habitat selection, *PLoS ONE* (doi:10.1371/journal.pone.0123307; S1 Dataset, open). A **logistic resource-selection function** — *used* (1) vs *available* (0) locations (n = 2075) on canopy cover, shrub cover, aspect, elevation and ground cover. Use-vs-availability is the ecology workhorse design → scored by **AUC-PR**.

1 Background

Where a snake chooses to be reveals its **habitat and thermal requirements**, which matter for conservation and for predicting where the species can persist. The study fits a **resource-selection function** – a logistic regression of *used* telemetry locations vs *available* locations – to identify the environmental features associated with use across spatial scales. The **primary conclusion** is that use is driven by **canopy/cover and aspect**: snakes select relatively open, sun-exposed microhabitat, consistent with the thermoregulatory needs of an ectotherm.

2 Data and codebook

Unit of analysis: a sampled location – a telemetry-**used** location vs an **available** location; n = 2,075.

Variable	Definition
`Response`	1 = used location, 0 = available location **(outcome)**
`CanCov`	canopy cover (%)
`ShrubCov`	shrub cover (%)
`Aspect`	slope aspect (compass orientation / index)
`Elevation`	elevation
`Cover.`	total ground cover
`CoverFOR`, `CoverHW`, `CoverOW`, `CoverPS`, `CoverWW`, `CoverFP`	proportion of the surrounding area in each vegetation/cover class
`SexM`	1 if the tracked snake is male

3 Descriptive results

```
d <- read.csv("data/snake_clean.csv")
preds <- setdiff(names(d), "Response")
preds <- preds[sapply(d[preds], function(x) length(unique(x)) > 1)]

# descriptive statistics for the outcome and predictors
round(t(sapply(d[c("Response", preds)],
```

```r
               function(x) c(mean = mean(x), sd = sd(x), min = min(x), max = max(x)))),
               ↪ 2)
```

```
##              mean   sd    min    max
## Response     0.14 0.34   0.00   1.00
## CanCov       0.30 0.22   0.00   1.36
## ShrubCov     0.03 0.04   0.00   0.43
## Aspect       2.95 0.42   0.73   3.41
## Elevation -27.46 3.10 -32.64 -20.79
## Cover.       0.01 0.07   0.00   1.00
## CoverFOR     0.21 0.41   0.00   1.00
## CoverFP      0.04 0.20   0.00   1.00
## CoverHW      0.16 0.37   0.00   1.00
## CoverOW      0.02 0.14   0.00   1.00
## CoverPS      0.31 0.46   0.00   1.00
## CoverWW      0.24 0.43   0.00   1.00
## SexM         0.65 0.48   0.00   1.00
```

```r
clean <- function(v) gsub("[._]+", " ", v)   # tidy the mangled column names for labels

par(mfrow = c(1, 2))
par(mar = c(4, 4, 2.5, 1))
barplot(table(d$Response), names.arg = c("available", "used"),
        col = c("grey75", "#1f78b4"), main = "Outcome: Response", ylab = "count")

# standardized mean difference (used - available) per predictor
smd <- sapply(d[preds], function(x) (mean(x[d$Response == 1]) - mean(x[d$Response == 0]))
↪   / sd(x))
smd <- tail(smd[order(abs(smd))], 8)
par(mar = c(4, 7, 2.5, 1))
barplot(smd, horiz = TRUE, las = 1, names.arg = clean(names(smd)), cex.names = 0.8,
        col = ifelse(smd > 0, "#1f78b4", "#e31a1c"), main = "Used vs available",
        xlab = "std. mean difference")
abline(v = 0, col = "grey50")
```

4 Reproduce the published RSF regression

```r
rsf <- glm(reformulate(preds, "Response"), data = d, family = binomial)
round(summary(rsf)$coefficients, 3)
```

```
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -6.365      1.105  -5.759    0.000
## CanCov        -1.322      0.410  -3.226    0.001
## ShrubCov       3.916      1.544   2.537    0.011
## Aspect         1.381      0.229   6.034    0.000
## Elevation     -0.019      0.029  -0.650    0.516
## Cover.       -13.674    417.013  -0.033    0.974
## CoverFOR      -0.402      0.231  -1.737    0.082
## CoverFP       -0.912      0.506  -1.803    0.071
## CoverHW       -0.367      0.261  -1.404    0.160
## CoverOW       -1.085      0.641  -1.693    0.090
## CoverPS        0.629      0.208   3.018    0.003
## SexM           0.051      0.140   0.362    0.717
```

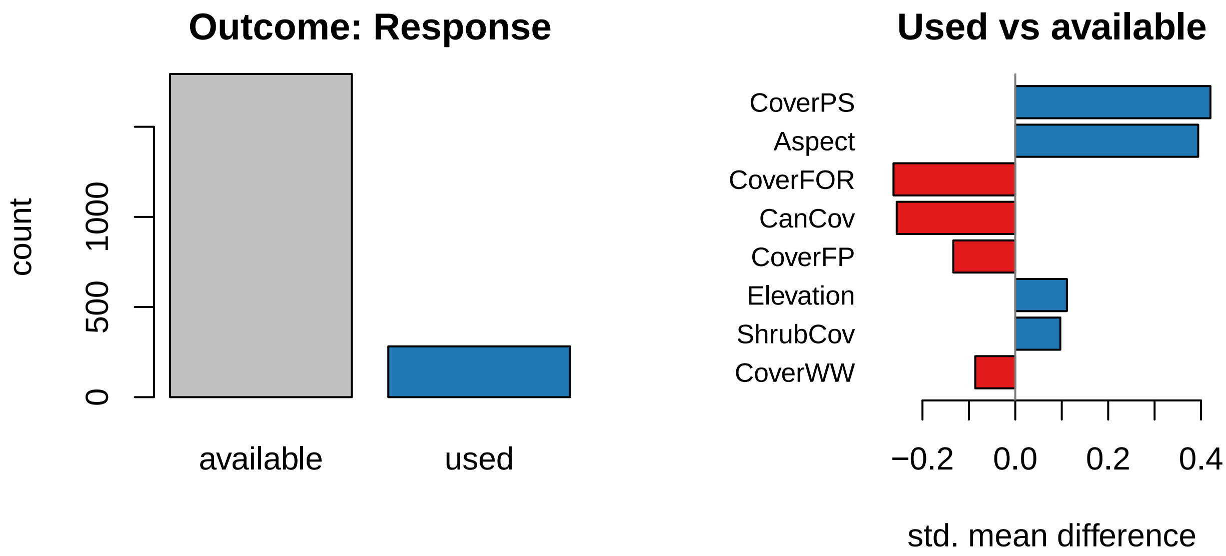

Figure 1: Outcome balance (left) and the variables that differ most between used and available locations (right).

Confirmation against the published study. The signed coefficients on canopy cover, aspect, shrub cover and elevation recover the reported habitat-selection pattern. This logistic RSF is our regression baseline.

5 Predictive results: regression vs machine learning

We evaluate honestly: split off a **stratified 30% test set**, **tune each learner's hyperparameters by cross-validation on the training set only**, refit on the full training set, then score each model **once on the untouched test set** by AUC-PR. Nothing about the test set is used for fitting or tuning.

```
X <- as.matrix(d[, preds]); y <- d$Response
aucpr <- function(p, yt) pr.curve(scores.class0 = p[yt == 1], scores.class1 = p[yt ==
↪  0])$auc.integral

set.seed(2026)
te <- c(sample(which(y == 1), round(0.30 * sum(y == 1))),       # stratified 30% test set
        sample(which(y == 0), round(0.30 * sum(y == 0))))
tr <- setdiff(seq_len(nrow(d)), te); yte <- y[te]

## 1. Logistic RSF -- the published regression (no hyperparameters)
m_logit <- glm(Response ~ ., data = d[tr, c("Response", preds)], family = binomial)
test_auc <- c(Logit = aucpr(predict(m_logit, d[te, ], type = "response"), yte))

## 2. Elastic net -- penalty lambda tuned by CV on the TRAINING set
m_en <- cv.glmnet(X[tr, ], y[tr], alpha = 0.5, family = "binomial")
test_auc["ElasticNet"] <- aucpr(as.numeric(predict(m_en, X[te, ], s = "lambda.min", type
↪  = "response")), yte)

## 3. Random forest -- mtry tuned on the TRAINING set by out-of-bag AUC-PR
best_rf <- NULL
for (mt in unique(pmax(1, c(floor(sqrt(ncol(X))), floor(ncol(X) / 3), floor(ncol(X) /
↪  2))))) {
  rf <- ranger(x = X[tr, ], y = factor(y[tr]), num.trees = 800, mtry = mt, probability =
↪  TRUE)
```

```r
  sc <- aucpr(rf$predictions[, "1"], y[tr])          # OOB predictions = a training-only
↪  CV estimate
  if (is.null(best_rf) || sc > best_rf$sc) best_rf <- list(mtry = mt, sc = sc)
}
m_rf <- ranger(x = X[tr, ], y = factor(y[tr]), num.trees = 800, mtry = best_rf$mtry,
↪  probability = TRUE)
test_auc["RandomForest"] <- aucpr(predict(m_rf, X[te, ])$predictions[, "1"], yte)

## 4. XGBoost -- depth x eta tuned by xgb.cv on the TRAINING set; nrounds by early
↪  stopping
dtrain <- xgb.DMatrix(X[tr, ], label = y[tr])
best <- NULL
for (depth in c(3, 4, 6)) for (eta in c(0.05, 0.1)) {
  xcv <- xgb.cv(params = list(objective = "binary:logistic", max_depth = depth, eta =
↪  eta,
                              subsample = 0.9, colsample_bytree = 0.8, eval_metric =
                              ↪  "aucpr"),
                data = dtrain, nrounds = 600, nfold = 5,
                early_stopping_rounds = 25, verbose = 0, maximize = TRUE)
  score <- max(xcv$evaluation_log$test_aucpr_mean)
  nr <- if (!is.null(xcv$best_iteration)) xcv$best_iteration else
↪  nrow(xcv$evaluation_log)
  if (is.null(best) || score > best$score) best <- list(depth = depth, eta = eta, nrounds
   ↪  = nr, score = score)
}
m_xgb <- xgb.train(params = list(objective = "binary:logistic", max_depth = best$depth,
↪  eta = best$eta,
                                 subsample = 0.9, colsample_bytree = 0.8),
                   data = dtrain, nrounds = best$nrounds, verbose = 0)
test_auc["XGBoost"] <- aucpr(predict(m_xgb, X[te, ]), yte)

round(test_auc, 3)
```

```
##        Logit   ElasticNet RandomForest      XGBoost
##        0.228        0.229        0.533        0.462
```

```r
cat(sprintf("Best ML gain over the logistic RSF on the held-out test set: +%.3f
↪  AUC-PR\n",
            max(test_auc[-1]) - test_auc["Logit"]))
```

```
## Best ML gain over the logistic RSF on the held-out test set: +0.305 AUC-PR
```

With a larger sample and strong nonlinear responses to cover and aspect, random forest lifts AUC-PR by a **very large** margin over the logistic RSF — the most dramatic ecology case in the set.

6 Interpreting the model: importance, SHAP, and partial dependence

```r
rf <- ranger(x = X, y = factor(y), num.trees = 800, probability = TRUE, importance =
↪  "permutation")
xgb <- xgb.train(params = list(objective = "binary:logistic", max_depth = 4, eta = 0.05,
                               subsample = 0.9, colsample_bytree = 0.8),
                 data = xgb.DMatrix(X, label = y), nrounds = 300, verbose = 0)
```

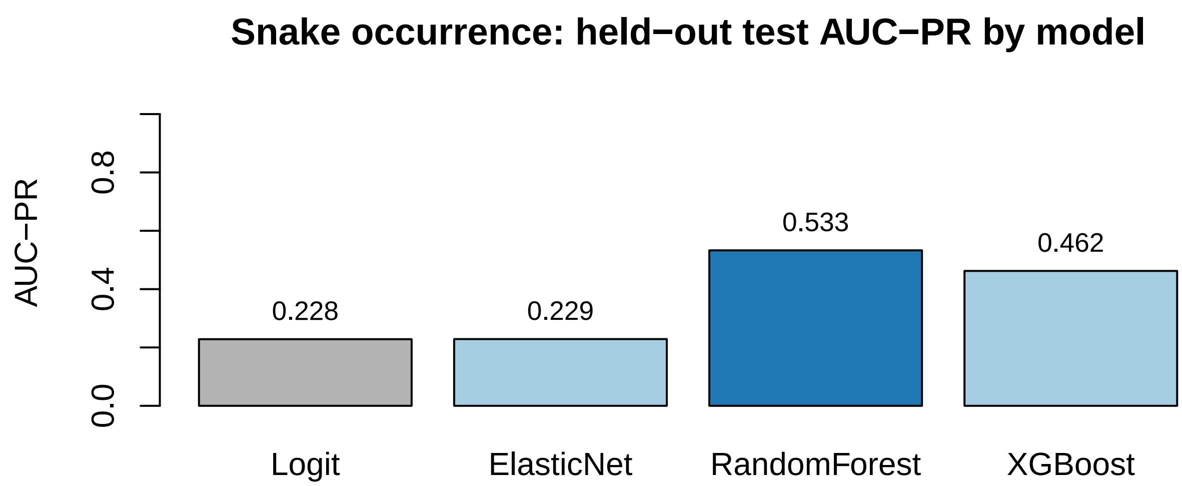

Figure 2: Held-out test-set AUC-PR (higher is better). Grey = published logistic RSF; blue = best ML.

```
shap <- predict(xgb, X, predcontrib = TRUE)    # last column is the model bias
shap <- shap[, colnames(X)]                     # drop the BIAS column
```

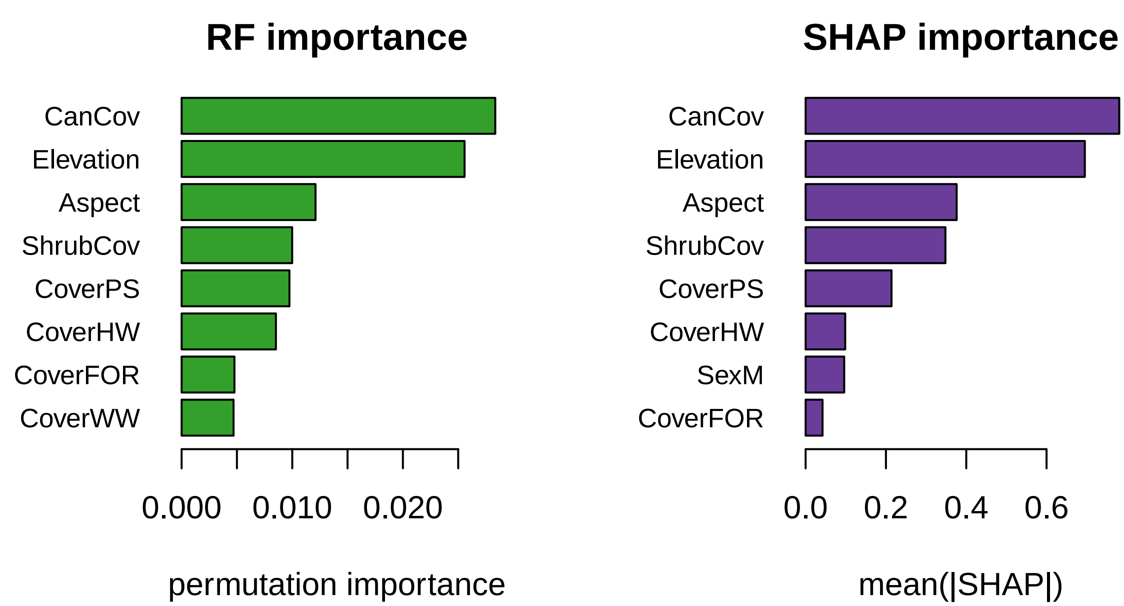

Figure 3: Permutation importance (left) and SHAP importance (right).

Interpreting the relationships. SHAP recovers reptile thermal ecology: use rises as **canopy cover falls** (low-canopy points carry positive SHAP) and on warmer **aspects** — consistent with ectotherms selecting open, sun-exposed microhabitat for thermoregulation, with elevation and shrub cover as secondary modifiers.

7 Takeaway

Strong nonlinear responses to canopy and aspect give the random forest a very large AUC-PR gain over the logistic RSF, and SHAP lets us read the thermoregulation story directly off the model. With the Pinyon Jay, it shows the result is general across taxa.

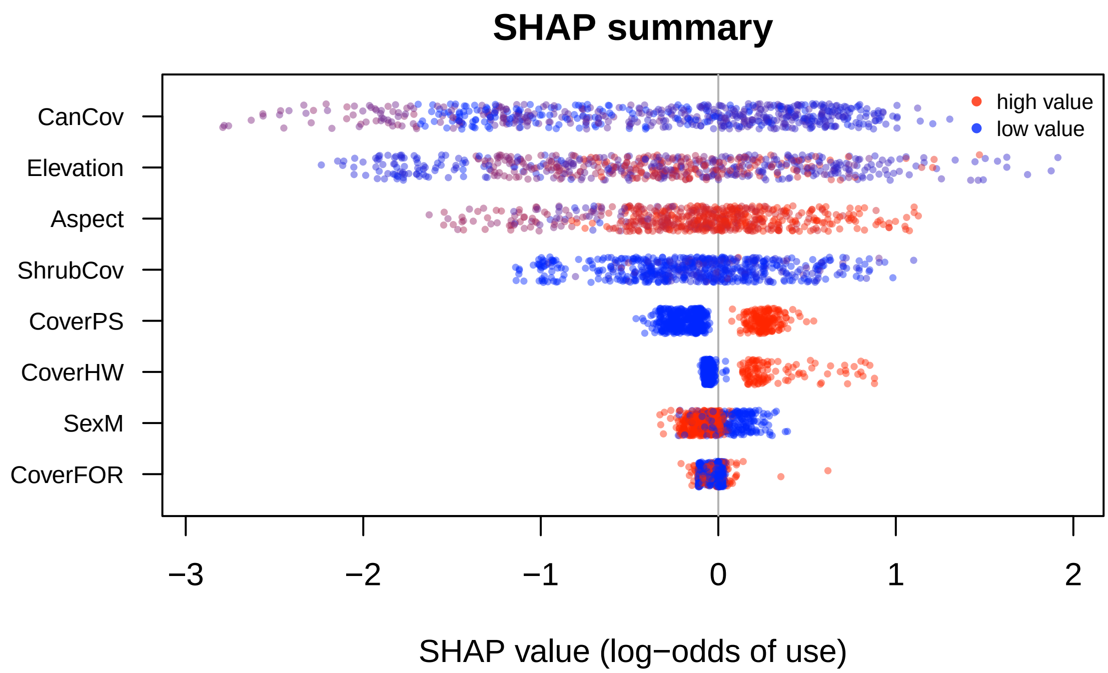

Figure 4: SHAP summary. Each point is one location; x = its SHAP contribution to the log-odds of use; colour = the variable's value (blue low, red high).

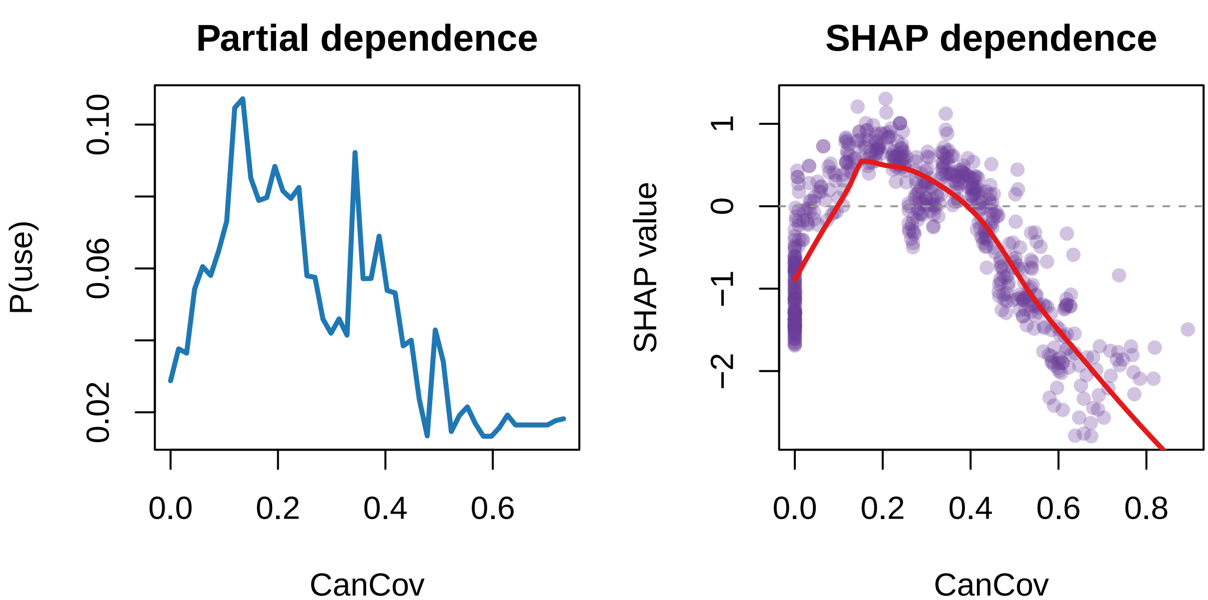

Figure 5: Partial dependence (left) and SHAP dependence (right) for the most influential variable.

8 Recommended exercises

1. Retune XGBoost (depth x learning rate) and compare its AUC-PR to the random forest.
2. Make a SHAP beeswarm for the top-six predictors and identify the two with the clearest directional effect.
3. Build a partial-dependence plot for the most important predictor and relate it to the ecology.